

Critical Path in Stochastic Network

LCTA 演算法可協助隨機網路找出 Critical Path，本文將該演算稱為” Critical Path Analysis, CPA”，CPA 方法也很簡單，只要在網路結構中，找到需要進行 max 運算的節點，並在該節點處進行”子節點路徑的重新排序”，就能夠完成 Critical Path 的演算。

以圖 1 為範例說明：

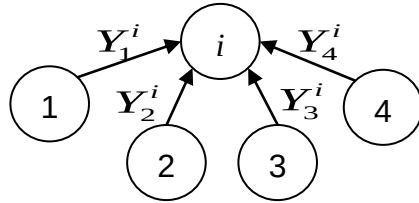


圖 1: 4 incident paths of node(i)

網路節點 i 的 $Prec_Node(i) = \{1,2,3,4\}$ ，共計 4 個輸入節點路徑： $Y_1^i \sim Y_4^i$ ，且均為隨機變數，在 LCTA 演算中，節點 i 若完成所有節點路徑的時間累積計算，接續便執行 max 運算，CPA 要求運算前須先進行”子節點路徑的重新排序”，排序的完整流程如下：

1. 先計算每條路徑期望值的大小 $E(Y_1^i) \sim E(Y_4^i)$ ，並由”左(小)→右(大)”做排序，假設： $E(Y_2^i) > E(Y_3^i) > E(Y_1^i) > E(Y_4^i)$ ，則圖 1 須調整為圖 2：

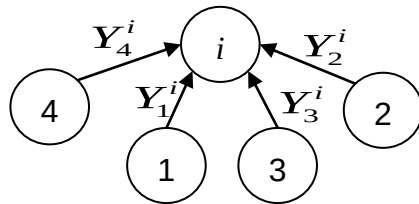


圖 2

2. 順序確定後，節點 i 內部若干資料結構串列資料內的”相對應順序位置”也需要同步變動：

- $Prec_Node(i)$
- $Path_Flag(i)$ 旗標
- $Path_Time(i)$

本文將上述排序的執行步驟以一函式表示： $SDO(i)$ ， $SDO(i)$ 執行後，會改變節點輸入路徑位置 → 改變網路樹狀結構形狀 → 改變網路節點連線的排列形狀，圖 3 為例，若 $SDO(8)$ 運算結果 $E(Y_6^8) > E(Y_7^8)$ ，節點 6 會安排在最右邊，反之，節點 7 會安排在最右

邊，整個網路樹狀結構都會跟著改變。

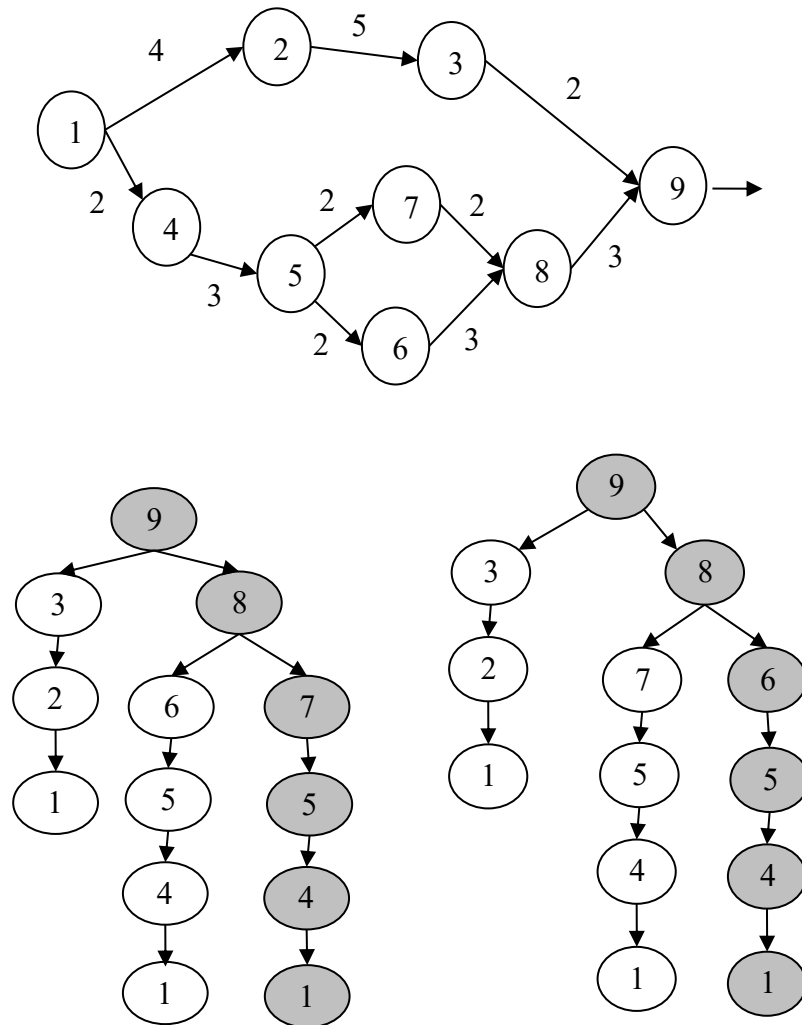


圖 3. SDO(i)函示範例說明

隨機網路關鍵路徑演算法(CPA)：

本文建立的 CPA 演算法，目的就是針對隨機網路找出關鍵路徑(Critical Path)，關鍵路徑可以是“最長時間路徑(longest duration path)”，也可以是“最短時間路徑(shortest duration path)”。

執行步驟如下所述：

1. 將上述**SDO(i)**運算功能導入進 LCTA 演算法，導入的位置：Upward_Tracing 函式內，執行節點 max 運算前，如下列所列

```

Downward_Tracing (j)          // j = current node
while (finish_flagN-1 = 0) do
    if (finish_flagj = 1)      // 節點 j 已完成計算?
        Shared_flag = 1;      // 節點 j 完成計算，標註“已被拜訪”
        k = j;                // last_node k = j
        j = pull_stack();      // 取出新 current_node 持續往上追蹤
    
```

$Path_Flag(j) = \{flag_k = 1\}, k \in B_j$

執行 **Upward_Tracing**(k, j) ;

Else

if ($j = 1$) // 碰觸起始節點?

$Output_j = [0, 1]$; // 定義起始節點為隨機變數 0

$finish_flag_j = 1$;

$k = j$; // last_node $k = j$

$j = pull_stack()$; // 取出新 current_node 持續往上追蹤

$Path_Flag(j) = \{flag_k = 1\}, k \in B_j$

執行 **Upward_Tracing**(k, j) ;

Else

$push_stack(j)$; // 將 j 至入堆疊中

由左至右順序，找出節點 j 下一個追蹤節點 (根據 $Path_Flag(j) = \{flag_k\}, k \in B_j$ ，若 $flag_k = 0$ 則選擇節點 k)

current_node $j = i$;

End

End

End

Upward_Tracing(k, j) // $k = last_node, j = current_node$

while ($finish_flag_{N-1} = 0$) **do**

if ($Shared_flag = 1$) // 表示 last_node k 已經被拜訪過

$Y_k^j = \mu(Output_k)$; // 去除變異數

$Shared_flag = 0$; // Reset $Shared_flag$ value

Else

$Y_k^j = Output_k$;

$Path_Flag(j) = \{flag_k = 1\}, k \in B_j$;

End

if (Y_k^j 的樣本單元大於 100)

$DRT(Y_k^j)$; // 執行重新取樣計算工作

End

if ($finish_flag_j = 0$) // 節點 j 尚未完成計算?

$push_stack(j)$; // 將 current_node j 至入堆疊中

由左至右順序，找出節點 j 下一個追蹤節點 (根據 $Path_Flag(j) = \{flag_k\}, k \in B_j$ ，若 $flag_k = 0$ 則選擇節點 k)

current_node $j = i$;

執行 **Downward_Tracing** (j) ;

```

Else                                     // 完成節點  $j$  的計算
    SDO( $i$ ) ;      // 輸入路徑  $\{Y_k^i | \forall k \in B_i\}$  重新排序
     $Output_j = \max\{Y_i^j\} \oplus \text{Node\_Time}(j), i \in B_j$  ;
     $finish\_flag_j = 1$  ;
     $k = j$ ;                               // last_node  $k = j$ 
     $j = \text{pull\_stack}()$  ;           // 取出新 current_node 持續往上追蹤
End
End

```

2. 當上述 LCTA 演算法運算完畢，網路樹狀結構的”最右邊沿線的節點路徑”，就是”最長時間路徑(longest duration path)”，以圖 2 為例，若 $E(Y_6^8) > E(Y_7^8)$ ， $\langle 1, 4, 5, 6, 8, 9 \rangle$ 即為最長路徑
3. 若 **SDO**(i) 改換規則：由左自右 $\rightarrow$ 由大而小排序($E(Y_k^i)$ 數值愈大的路徑，安排在最左邊位置)，網路樹狀結構的”最右邊沿線的節點路徑”，就是”最短時間路徑(shortest duration path)”，LCTA 演算內容完全不用做額外改變
4. 特別提醒：**SDO**(i) 函式進行輸入路徑的位置調整，也會牽動 LCTA 演算法中使用的 **Path_Flag**(i) 旗標及 **Path_Time**(i) 內資訊，其”相對應順序位置”需要同步更改